

DOSSIER DE CONCEPTION

Application de Gestion de Bibliothèque Universitaire

Bibliothèque UNZ — Système de Gestion Documentaire

Équipe de Développement

<i>Nom et Prénom(s)</i>	<i>Rôle</i>
<i>BANAO Moumouni</i>	<i>(Scrum Master)</i>
<i>KIINDO Youba</i>	<i>(Product Owner)</i>
<i>PORGO Zakaria</i>	<i>Developper</i>
<i>BAYALA Eric</i>	<i>Developper</i>
<i>SANKARA Oumarou</i>	<i>Developper</i>
<i>DAHANI Michael</i>	<i>Developper</i>
<i>YAMEOGO A. Laurent</i>	<i>Developper</i>

1. Cahier des Charges

1.1 Contexte et Objectifs

L'Université Norbert Zongo souhaite moderniser sa bibliothèque en développant une application web complète de gestion documentaire. Cette application vise à remplacer les processus manuels par un système numérique fiable, sécurisé et accessible.

L'objectif principal est de concevoir, modéliser, développer et tester une application logicielle professionnelle en appliquant les concepts du Génie Logiciel : principes SOLID, Design Patterns, tests unitaires, méthodologie Agile Scrum.

1.2 Acteurs du Système

Acteur	Rôle et responsabilités
Étudiant	Consulte le catalogue, recherche des ouvrages, emprunte, réserve, consulte son historique et reçoit des notifications
Bibliothécaire	Gère le catalogue, les emprunts, les retours, les pénalités et les notifications
Administrateur	Gère les utilisateurs, les statistiques, les configurations du système et les rapports

1.3 Exigences Fonctionnelles

#	Exigence	Description
EF1	Gestion du catalogue	Ajout, modification, suppression d'ouvrages, auteurs, exemplaires
EF2	Recherche avancée	Par titre, auteur, ISBN, mots-clés, catégorie
EF3	Emprunt et retour	Enregistrement des emprunts et retours avec mise à jour des stocks
EF4	Réservation	Réservation quand tous les exemplaires sont empruntés
EF5	Pénalités de retard	Calcul automatique selon le nombre de jours de retard
EF6	Notifications automatiques	Email pour confirmation, rappel J-2, retard, disponibilité
EF7	Historique des emprunts	Consultation par l'étudiant de ses emprunts passés et en cours
EF8	Gestion des utilisateurs	Inscription, authentification JWT, profils par rôle
EF9	Statistiques et rapports	Ouvrages populaires, taux de retard, emprunts par mois
EF10	Catégories et rayons	Organisation thématique des ouvrages par catégorie et rayon

1.4 Exigences Non Fonctionnelles

Exigence	Détail
Sécurité	Authentification JWT, hachage BCrypt, protection des routes par rôle
Interface responsive	Accessible sur ordinateur et mobile, CSS adaptif
Performance	Temps de réponse < 2 secondes pour les opérations courantes
Disponibilité	Disponibilité >= 99% (hébergement Render)
Maintenabilité	Respect des principes SOLID, utilisation d'au moins 4 Design Patterns
Versioning	Code versionné avec Git, dépôt GitHub public
Testabilité	Couverture de tests >= 65% (JaCoCo)

1.5 Users Stories

ID	Acteur	En tant que... je veux...	Afin de...
US-01	Étudiant	m'inscrire avec mon email Gmail et mon INE	accéder à la plateforme en ligne
US-02	Utilisateur	me connecter avec email et mot de passe	accéder à mon espace personnel sécurisé
US-03	Admin	gérer les ouvrages du catalogue	maintenir la bibliothèque à jour
US-04	Étudiant	consulter le catalogue avec filtres	trouver rapidement un ouvrage
US-05	Étudiant	rechercher par titre, auteur, ISBN ou catégorie	affiner mes recherches
US-06	Biblio.	enregistrer un emprunt	tracer les sorties d'ouvrages
US-07	Biblio.	enregistrer un retour	remettre l'ouvrage en disponibilité
US-08	Étudiant	réserver un ouvrage indisponible	être prioritaire à sa disponibilité
US-09	Système	envoyer des notifications email	informer les étudiants automatiquement
US-10	Admin	consulter les statistiques et rapports	piloter efficacement la bibliothèque

2. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation représente les interactions entre les acteurs du système (Étudiant, Bibliothécaire, Administrateur) et les fonctionnalités offertes par l'application.

2.1 Acteurs et cas d'utilisation

Acteur	Cas d'utilisation
Étudiant	S'inscrire, Se connecter, Consulter catalogue, Rechercher ouvrage, Réserver, Consulter historique, Recevoir notifications, Annuler réservation
Bibliothécaire	Se connecter, Gérer catalogue, Enregistrer emprunt, Enregistrer retour, Calculer pénalités, Gérer catégories
Administrateur	Tous les cas du Bibliothécaire + Gérer utilisateurs, Consulter statistiques, Générer rapports, Créer comptes

Note : Le diagramme UML complet est disponible en annexe et dans l'outil GitHub Projects.

3. Diagramme de Classes

Le diagramme de classes décrit la structure statique de l'application, les entités du domaine, leurs attributs, méthodes et relations.

3.1 Classes principales

Classe	Attributs principaux	Méthodes principales
Utilisateur (abstract)	id, nom, prenom, email, motDePasse, role, dateInscription	Classe abstraite — héritage par Etudiant, Bibliothecaire, Administrateur
Etudiant	numeroEtudiant (INE), filiere	getEmprunts(), getReservations()
Bibliothecaire	matricule, service	gererEmprunts(), calculerPenalite()
Administrateur	niveauAcces	gererUtilisateurs(), genererRapports(), voirStatistiques()
Ouvrage	titre, auteur, isbn, description, anneePublication, nombreExemplaires, exemplairesDisponibles, categorie	isDisponible(), decrementer(), incrementer()
Categorie	nom, description, rayon	getOuvrages()
Emprunt	dateEmprunt, dateRetourPrevue, dateRetourReelle, statut	calculerPenalite(), estEnRetard(), retourner()
Reservation	dateReservation, statut	annuler(), confirmer()

3.2 Relations entre classes

Relation	Type	Cardinalité
Utilisateur → Etudiant	Héritage	Un Utilisateur peut être un Etudiant (spécialisation)
Utilisateur → Bibliothécaire	Héritage	Un Utilisateur peut être un Bibliothécaire
Utilisateur → Administrateur	Héritage	Un Utilisateur peut être un Administrateur
Etudiant → Emprunt	Association	Un Etudiant peut avoir 0..* Emprunts
Etudiant → Reservation	Association	Un Etudiant peut avoir 0..* Réservations
Ouvrage → Emprunt	Composition	Un Ouvrage peut avoir 0..* Emprunts
Ouvrage → Reservation	Association	Un Ouvrage peut avoir 0..* Réservations
Categorie → Ouvrage	Association	Une Catégorie contient 1..* Ouvrages

4. Diagrammes de Séquence

4.1 Séquence — Emprunt d'un ouvrage

Ce diagramme décrit le flux d'interactions lors de l'enregistrement d'un emprunt par un bibliothécaire.

#	Message	Description
1	creerEmprunt(idEtu, idOuv)	Le bibliothécaire saisit les IDs dans admin.html
2	POST /api/emprunts	Requête HTTP avec token JWT dans l'en-tête Authorization
3	creerEmprunt(etu, ouv)	Le contrôleur délègue au service métier
4	findById(ouvrageId)	Vérification de la disponibilité en base
AL T	[indisponible]	Exception levée : RuntimeException("Aucun exemplaire disponible")
5	save(ouvrage--)	Décrémentation du nombre d'exemplaires disponibles
6	save(emprunt)	Sauvegarde de l'emprunt avec statut EN_COURS
7	envoyerConfirmationEmprunt()	Email de confirmation envoyé à l'étudiant
8	200 OK + JSON	Réponse avec l'objet Emprunt créé

4.2 Séquence — Authentication JWT

Ce diagramme décrit le processus de connexion et de génération du token JWT.

#	Message	Description
1	Saisir email + mdp	L'utilisateur remplit le formulaire login.html
2	POST /api/auth/login	Envoi des credentials en JSON
3	findByEmail(email)	Recherche de l'utilisateur en base
AL T	[non trouvé]	Retour 401 : Email introuvable
4	matches(mdp, hash)	Vérification BCrypt du mot de passe
AL T	[incorrect]	Retour 401 : Mot de passe incorrect
5	genererToken(email, role)	Génération du token JWT signé HS256, valable 24h
6	{token, role, nom, id}	Réponse 200 avec token et informations utilisateur
7	localStorage.setItem()	Stockage du token côté client
8	Redirection	Vers admin.html ou etudiant.html selon le rôle

4.3 Séquence — Réservation d'un ouvrage

Ce diagramme décrit le processus de réservation d'un ouvrage indisponible par un étudiant.

#	Message	Description
1	Clic 'Réserver'	L'étudiant clique sur le bouton réserver dans etudiant.html
2	POST /api/reservations	Envoi de {etudiantId, ouvrageId} avec token JWT
3	existsByEtudiantIdAndOuvrageId()	Vérification qu'une réservation n'existe pas déjà
AL T	[déjà réservé]	Exception : Vous avez déjà réservé cet ouvrage
4	save(reservation)	Sauvegarde avec statut EN_ATTENTE
5	envoyerConfirmationReservation()	Email de confirmation envoyé à l'étudiant
6	200 OK + JSON	Réponse avec la réservation créée

5. Conception Architecturale (4+1 Vues)

L'architecture de l'application est décrite selon le modèle 4+1 vues de Philippe Kruchten : vue logique, vue développement, vue processus, vue physique et vue scénarios.

5.1 Vue Logique — Architecture en couches

L'application suit une architecture MVC en 4 couches distinctes :

Couche	Technologie	Responsabilité
Présentation	HTML5 / CSS3 / JavaScript	Interface utilisateur, formulaires, graphiques Chart.js
Contrôleur	Spring Boot REST (@RestController)	Exposition des endpoints API, validation des requêtes
Service	Spring Service (@Service)	Logique métier, règles d'application, calculs
Repository	Spring Data JPA (@Repository)	Accès base de données, requêtes JPQL, MySQL

5.2 Vue Développement — Structure des packages

Package	Contenu
com.bibliotheque.controller	AuthController, EmpruntController, OuvrageController, ReservationController, CategorieController, StatistiqueController, UtilisateurController
com.bibliotheque.service	EmpruntService, OuvrageService, ReservationService, NotificationService
com.bibliotheque.model	Utilisateur, Etudiant, Bibliothecaire, Administrateur, Ouvrage, Categorie, Emprunt, Reservation, Role, StatutEmprunt, StatutReservation
com.bibliotheque.repository	EmpruntRepository, OuvrageRepository, ReservationRepository, UtilisateurRepository, CategorieRepository
com.bibliotheque.config	SecurityConfig, JwtUtil, JwtFilter, DatabaseConfig
com.bibliotheque.strategy	CalculPenalite (interface), PenaliteStandard, PenaliteBoursier
com.bibliotheque.factory	UtilisateurFactory
com.bibliotheque.observer	NotificationObserver (pattern Observer via NotificationService)

5.3 Vue Processus — Flux d'exécution

Chaque requête HTTP suit ce flux de traitement :

1. La requête arrive sur le serveur Tomcat embarqué (port 8081)
2. Le JwtFilter intercepte la requête, extrait et valide le token Bearer

3. Spring Security vérifie les permissions selon le rôle (ETUDIANT, BIBLIOTHECAIRE, ADMINISTRATEUR)
4. Le Controller approprié traite la requête et délègue au Service
5. Le Service applique la logique métier et utilise le Repository
6. Le Repository exécute la requête JPA sur MySQL
7. La réponse remonte en JSON avec le code HTTP approprié

Tâches planifiées (@Scheduled) :

- Chaque matin à 8h : envoi des rappels de retour J-2
- Chaque matin à 9h : détection et marquage des emprunts en retard

5.4 Vue Physique — Déploiement

Composant	Description
Navigateur Client	Chrome, Firefox, Safari — charge login.html, admin.html, etudiant.html depuis le serveur
Serveur Spring Boot	Hébergé sur Render (plan gratuit), port 8081, JRE 21, Tomcat embarqué
Base de données	MySQL 8.0, hébergée sur PlanetScale ou Railway (plan gratuit)
Serveur SMTP	Gmail SMTP (smtp.gmail.com:587) pour l'envoi des notifications email

5.5 Vue Scénarios — Cas d'utilisation clés

Les scénarios principaux illustrant l'architecture en action :

- S1 — Emprunt : Bibliothécaire → admin.html → POST /api/emprunts → EmpruntService → MySQL + Email
- S2 — Réservation : Étudiant → etudiant.html → POST /api/reservations → ReservationService → MySQL + Email
- S3 — Authentification : Utilisateur → login.html → POST /api/auth/login → JwtUtil → JWT Token
- S4 — Notification : @Scheduled → EmpruntService → NotificationService → Gmail SMTP
- S5 — Pénalités : Retard détecté → PenaliteStandard.calculer() → Pattern Strategy
- S6 — Statistiques : Admin → admin.html → GET /api/statistiques → Chart.js

6. Justification des Design Patterns

Conformément aux exigences du projet, quatre Design Patterns ont été implémentés et justifiés ci-dessous.

6.1 Pattern Strategy — Calcul des Pénalités

Aspect	Détail
Problème	Le calcul des pénalités peut varier selon le profil de l'étudiant (standard, boursier, etc.)
Solution	Interface CalculPenalite avec implémentations PenaliteStandard (100 FCFA/jour) et PenaliteBoursier (50 FCFA/jour)
Fichiers	CalculPenalite.java (interface), PenaliteStandard.java, PenaliteBoursier.java, EmpruntService.java
Avantage	Facile d'ajouter de nouvelles stratégies sans modifier le code existant (principe Open/Closed — SOLID)

6.2 Pattern Singleton — Gestion des Beans Spring

Aspect	Détail
Problème	Garantir une seule instance des services et repositories dans toute l'application
Solution	Spring IoC Container gère le cycle de vie des Beans en Singleton par défaut (@Service, @Repository, @Component)
Fichiers	DatabaseConfig.java, tous les @Service et @Repository
Avantage	Économie de ressources, cohérence de l'état, gestion automatique par Spring

6.3 Pattern Factory — Création des Utilisateurs

Aspect	Détail
Problème	La création d'utilisateurs (Etudiant, Bibliothecaire, Administrateur) varie selon le type
Solution	UtilisateurFactory avec méthode statique creerUtilisateur(type, nom, email) qui instancie le bon type
Fichiers	UtilisateurFactory.java, FactoryTest.java
Avantage	Centralise la logique de création, facilite l'extension pour de nouveaux types d'utilisateurs

6.4 Pattern Observer — Système de Notifications

Aspect	Détail
Problème	Les événements (emprunt, retour, réservation) doivent déclencher automatiquement des notifications
Solution	NotificationService observe les événements du système et envoie les emails correspondants via JavaMailSender
Fichiers	NotificationService.java, EmpruntService.java, ReservationService.java
Avantage	Découplage entre la logique métier et les notifications, facile d'ajouter de nouveaux canaux (SMS, push)

7. Technologies Utilisées

Technologie	Version	Utilisation
Spring Boot	3.2.0	Framework principal backend, REST API, IoC
Spring Security	3.2.0	Authentification, autorisation par rôle, filtres
Spring Data JPA	3.2.0	ORM, accès base de données, requêtes JPQL
MySQL	8.0	Système de gestion de base de données relationnelle
JWT (jjwt)	0.11.5	Génération et validation des tokens d'authentification
BCrypt	Spring	Hachage sécurisé des mots de passe
Spring Mail	3.2.0	Envoi des notifications email via Gmail SMTP
JUnit 5	5.x	Tests unitaires et d'intégration
Mockito	5.x	Simulation des dépendances dans les tests
JaCoCo	0.8.11	Mesure de la couverture de tests (74% obtenu)
Lombok	1.18.x	Génération automatique des getters/setters/constructeurs
Chart.js	4.4.0	Graphiques statistiques dans le tableau de bord
Java	21 (LTS)	Langage de programmation principal
Maven	3.x	Gestion des dépendances et build

8. Tests et Couverture

8.1 Résultats JaCoCo

Package	Couverture Instructions	Couverture Branches
com.bibliotheque.service	73%	31%
com.bibliotheque.factory	88%	100%

Package	Couverture Instructions	Couverture Branches
com.bibliotheque.strategy	100%	n/a
TOTAL	74% >= 65%	42%

8.2 Classes de tests

- EmpruntServiceTest — 6 tests : création, indisponibilité, retour, non trouvé, pénalité
- OuvrageServiceTest — 7 tests : CRUD, recherche par titre, disponibilité
- OuvrageServiceAvanceeTest — 7 tests : ISBN, auteur, catégorie, rayon, multi-critères
- ReservationServiceTest — 6 tests : création, doublon, annulation, non trouvé
- NotificationServiceTest — 7 tests : confirmation emprunt/réservation, rappel, retard, bienvenue
- CategorieTest — 6 tests : CRUD catégorie, association avec ouvrage
- PenaliteTest — 3 tests : standard, boursier, comparaison
- ModelTest — 7 tests : validation des entités
- FactoryTest — 4 tests : création des différents types d'utilisateurs

9. Conclusion

Ce dossier de conception présente l'architecture et la modélisation de l'application de gestion de bibliothèque universitaire de l'UNZ. Le système a été conçu en respectant les principes du Génie Logiciel :

- Architecture en couches (MVC) avec séparation claire des responsabilités
- 4 Design Patterns implémentés et justifiés (Strategy, Singleton, Factory, Observer)
- Sécurité robuste avec JWT et BCrypt
- Tests unitaires avec couverture JaCoCo de 74% (exigence : 65%)
- Méthodologie Agile Scrum sur 3 sprints avec livraisons itératives
- Notifications automatiques par email
- Interface responsive pour ordinateur et mobile

L'application est fonctionnelle et déployable sur Render. Le code source est disponible sur GitHub avec le Product Backlog et les artefacts Scrum.